SIMATS ENGINEERING
ASSESSMENT TOOL 2: Case Study

COURSE NAME: COMPUTER VISION COURSE CODE:ITA0515 Slot B

COURSE OUTCOME COVERED

CO3: ***Analyze and extract image features using detection and matching techniques for effective image representation and comparison.*** **(BTL 4)**

Assessment Tool Weightage: 40%

QUESTIONS:5 **Total Marks 50** **Duration :60 Minutes**

Annexure A

Case Study

Code of Conduct:

I M.Pranay (192525382) ***certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.***

Signature of the student: *M. Pranay*

Rubrics for Evaluation

Criteria	Weightae (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	*Marks(100)*
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly	
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts	
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning	
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided	
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand	

1. Preprocessing and Feature Detection

A vision system processes images captured under varying lighting conditions. Without preprocessing, feature detection results are inconsistent.Analyze the issue and explain how preprocessing improves feature detection performance.

2. Edge vs Corner Detection

In an image containing both straight edges and textured regions, a system uses edge detection but fails to identify key points for matching.Analyze why edge detection is insufficient and justify the use of corner detection in this case.

3. Hough Transform in Noisy Images

A system uses Hough Transform to detect lines in noisy images but produces inaccurate results.
Analyze the effect of noise on detection and suggest improvements based on preprocessing or parameter selection.

4. Feature Matching under Transformations

Two images of the same object are taken at different scales and orientations. Traditional feature matching fails, but scale invariant feature matching works effectively.Analyze why scale invariant methods perform better in this scenario.

5. PCA for Feature Optimization

A system extracts a large number of features, leading to high computational cost and redundancy.
Analyze how Principal Component Analysis helps in reducing dimensionality and improving efficiency.

ANSWER

1. Preprocessing and Feature Detection

Problem

A vision system captures images under different lighting conditions. If preprocessing is not performed, the same object may appear very different in different images. This causes feature detection algorithms to produce inconsistent results.

For example, an image captured in bright light may have high pixel intensities, while the same image captured in low light may appear dark. Shadows, highlights, and uneven illumination can also create unwanted features.

Analysis

Feature detection algorithms such as **Harris Corner, SIFT, ORB, and FAST** depend on image properties such as intensity changes, gradients, edges, and local textures.

Changes in lighting can affect these properties:

- Brightness changes pixel intensity.
- Shadows create additional intensity boundaries.
- Overexposure can remove important details.
- Low illumination can hide image features.
- Uneven lighting can produce false edges and corners.

Therefore, the same physical feature may not be detected consistently.

How Preprocessing Helps

Preprocessing improves the image before feature detection.

Common techniques include:

1. Grayscale conversion

Reduces a color image to intensity information and simplifies processing.

2. Histogram Equalization

Improves contrast and makes features more visible.

3. Gaussian Filtering

Reduces random noise and unwanted small variations.

4. Contrast Limited Adaptive Histogram Equalization (CLAHE)

Improves local contrast and is useful for images with uneven illumination.

5. Normalization

Adjusts intensity values to a common range.

Result

After preprocessing, important edges, corners, and textures become more consistent. This allows the feature detector to identify the same features under different lighting conditions.

Conclusion

Preprocessing is an important step in computer vision because it reduces the effect of lighting, noise, and contrast variations. It improves the **accuracy, stability, and repeatability** of feature detection.

2. Edge vs Corner Detection

Problem

An image contains straight edges as well as textured regions. The system uses edge detection but fails to identify important key points required for image matching.

Analysis

Edge detection identifies locations where there is a significant change in intensity.

Examples of edges include:

- Boundary of a building
- Side of a road
- Object outline
- Straight lines

Common edge detectors include:

- Sobel
- Prewitt

- Canny

However, an edge does not provide a unique point for matching.

For example:

EDGE

Many points along this edge have similar characteristics. Therefore, it is difficult to determine exactly which point corresponds to a point in another image.

Corner Detection

A corner is a location where intensity changes significantly in **two directions**.

For example:

|

|

|_____

The intersection provides a distinctive location.

Corner detectors include:

- Harris Corner Detector
- Shi-Tomasi
- FAST

Why Corners Are Better for Matching

Corners are more distinctive than ordinary edges because they contain strong changes in multiple directions.

For example:

Feature	**Edge Detection**	**Corner Detection**
Detects	Boundaries	Key points
Intensity change	Mainly one direction	Two or more directions

Feature	Edge Detection	Corner Detection
Uniqueness	Usually low	Usually high
Suitable for matching	Limited	Highly suitable
Example	Straight wall boundary	Building corner

Conclusion

Edge detection alone is insufficient because edges represent continuous boundaries rather than unique key points. **Corner detection identifies distinctive locations that can be reliably matched between images.** Therefore, corner detection is more appropriate when the objective is feature matching.

3. Hough Transform in Noisy Images

Problem

A system uses the **Hough Transform** to detect lines in noisy images but produces inaccurate results.

Analysis

The Hough Transform detects geometric shapes by converting image points into a parameter space.

For a straight line:

The Hough Transform commonly uses:

Each detected edge pixel votes for possible lines in Hough space. A large number of votes indicates a potential line.

Effect of Noise

Noise introduces unwanted intensity changes and false edges.

For example:

Original:

Noisy:

--.-.-.-------..----.------

The Hough Transform may interpret these unwanted pixels as parts of lines.

This can result in:

- False lines
- Multiple lines for one actual line
- Incorrect line positions
- Weak real lines being missed
- Increased computational cost

Improvements

1. Apply Gaussian Blur

Gaussian filtering reduces random noise before edge detection.

Noisy Image

↓

Gaussian Filter

↓

Canny Edge Detection

↓

Hough Transform

2. Use Median Filtering

Median filtering is useful for salt-and-pepper noise.

3. Use Canny Edge Detection

Canny can produce cleaner edges before applying the Hough Transform.

4. Adjust Hough Parameters

Important parameters include:

- **Threshold** – minimum number of votes required to detect a line.
- **Minimum line length** – minimum acceptable line length.
- **Maximum line gap** – maximum gap allowed between line segments.

Increasing the threshold can reduce false line detections.

Conclusion

Noise creates false edges that can accumulate votes in Hough space and produce inaccurate lines. Therefore, **noise removal followed by proper edge detection and suitable Hough parameter selection** improves line detection accuracy.

4. Feature Matching Under Transformations

Problem

Two images of the same object are captured at different **scales and orientations**. Traditional feature matching fails, while scale-invariant feature matching works effectively.

Analysis

Traditional feature descriptors may depend on the original size and orientation of the object.

For example:

Image 1: Image 2:

OBJECT OBJECT

↑ ↗

Normal Rotated

Small Large

The same feature may have a different appearance because of:

- Rotation
- Scaling
- Viewing angle

- Illumination changes

A traditional feature detector may therefore fail to identify corresponding points.

Scale-Invariant Feature Matching

Scale-invariant methods are designed to detect and describe features even when their size changes.

Examples include:

- **SIFT**
- **SURF**
- Some variants of ORB

SIFT, for example, detects features at different scales using a scale-space representation.

It also assigns an orientation to the feature. The descriptor is constructed relative to this orientation.

Therefore, a feature can still be recognized when:

- The object becomes larger.
- The object becomes smaller.
- The object is rotated.

Comparison

Feature	**Traditional Matching**	**Scale-Invariant Matching**
Scale changes	Poor performance	Good performance
Rotation	May fail	More robust
Feature correspondence	Less reliable	More reliable
Object recognition	Limited	Better
Suitable for transformed images	No/limited	Yes

Example

Suppose a logo appears in one image at 100% size and another image at 200% size.

A traditional descriptor may see them as significantly different.

A scale-invariant descriptor can identify that both features represent the **same underlying pattern**.

Conclusion

Scale-invariant feature matching performs better because it is designed to handle changes in feature size and orientation. It provides more reliable feature correspondence when images are captured under different transformations.

5. PCA for Feature Optimization

Problem

A system extracts a very large number of features. Many features contain similar or redundant information. Processing all these features increases computational cost.

Analysis

Suppose an image produces:

1000 features

↓

Many redundant features

↓

High memory usage

↓

High computation time

Principal Component Analysis (PCA) is a dimensionality reduction technique used to transform a large set of correlated features into a smaller set of important features.

How PCA Works

PCA identifies directions in the feature space that contain the greatest amount of variance.

The main steps are:

Step 1: Standardize the Data

Features are scaled so that differences in their numerical ranges do not dominate the analysis.

Step 2: Calculate Covariance Matrix

The covariance matrix identifies relationships between different features.

Step 3: Calculate Eigenvalues and Eigenvectors

The eigenvectors represent the principal directions, while the eigenvalues indicate how much variance each direction contains.

Step 4: Select Principal Components

Components with the highest eigenvalues are selected.

For example:

Original Features

↓

1000 Features

↓

PCA

↓

100 Important Components

Benefits of PCA

1. Reduces dimensionality

A large feature vector can be represented using fewer components.

2. Removes redundancy

Highly correlated information can be represented by fewer components.

3. Reduces computational cost

Machine learning and feature matching algorithms process fewer dimensions.

4. Reduces memory requirements

Smaller feature representations require less storage.

5. Can improve model performance

Removing irrelevant or redundant information can make classification or matching more efficient.

Example

Suppose a system extracts **500 features**.

After PCA:

The system now works with only 50 principal components while retaining most of the important information.

Conclusion

PCA improves feature optimization by reducing a large number of correlated or redundant features into a smaller set of informative components. This reduces **computational time, memory usage, and data complexity** while preserving most of the important information.